



گزارش پروژه شماره ۳: جستجوی خصمانه در بازی اتللو

لینک گیت هاب پروژه: <https://github.com/Elham-ch/AI-Projects>

معرفی مسئله

در این پروژه، بازی Othello به عنوان یک مسئله جستجوی خصمانه پیاده سازی شده است. در این بازی دو بازیکن با مهره های سیاه و سفید روی یک صفحه مربعی بازی می کنند. در کد پروژه، مقدار بازیکن سیاه برابر 1 و مقدار بازیکن سفید برابر -1 است. خانه خالی نیز با مقدار 0 نمایش داده می شود.

در هر حرکت، بازیکن باید مهره ای را در خانه ای قرار دهد که حداقل یک ردیف از مهره های حریف را بین مهره جدید و یکی از مهره های قبلی خودش محصور کند. مهره های محصور شده برگردانده می شوند. هدف نهایی این است که در پایان بازی تعداد مهره های بازیکن بیشتر از حریف باشد.

کلاس اصلی بازی در فایل `game/othello.py` قرار دارد. برای سازگاری با کد اولیه پروژه، عامل های جدید همان رابط عامل های آماده را دارند:

```
1 def choose_move(self, game, player):  
2     ...  
3     return move
```

به همین دلیل `MinimaxAgent` و `AlphaBetaAgent` می توانند بدون تغییر در ساختار کلی بازی، مانند `RandomAgent` و `GreedyAgent` در مسابقه اجرا شوند.

۱ توضیح روش پیاده سازی

برای پیاده سازی عامل ها از توابع موجود در فایل `othello.py` استفاده شده است. مهم ترین توابع و ویژگی هایی که عامل ها با آنها کار می کنند عبارت اند از:

- `game.get_valid_moves(player)` برای گرفتن حرکت های مجاز.
- `game.make_move(player, row, col)` برای اعمال حرکت.
- `game.copy()` برای ساختن نسخه مستقل از وضعیت بازی.
- `game.game_over()` برای تشخیص پایان بازی.
- `game.score()` برای گرفتن تعداد مهره های سیاه و سفید.
- `game.size` و `game.board` برای بررسی موقعیت های صفحه.

در جستجوی درخت بازی، هیچ حرکت فرضی روی خود بازی اصلی اعمال نمی شود. برای هر حرکت، ابتدا از وضعیت بازی کپی گرفته می شود و سپس حرکت روی آن کپی انجام می شود:

```
1 child = game.copy()  
2 child.make_move(player, *move)
```

این کار باعث می شود تابع `choose_move` و جستجوی داخلی عامل ها وضعیت واقعی بازی را تغییر ندهند.

۱.۱ عامل Minimax

الگوریتم Minimax در فایل `agents/minimax_agent.py` پیاده‌سازی شده است. ایده اصلی آن این است که بازیکن ریشه در گره‌های MAX سعی می‌کند مقدار ارزیابی را بیشینه کند و حریف در گره‌های MIN سعی می‌کند همان مقدار را کمینه کند. در این پیاده‌سازی، تابع `choose_move` فقط نقطه ورود عامل است. این تابع شمارنده گره‌های جستجو شده را صفر می‌کند، سپس جستجو را از تابع `max_value` شروع می‌کند:

```
1 def choose_move(self, game, player):
2     self.nodes_searched = 0
3     _, move = self.max_value(game, self.depth, player)
4     return move
```

در تابع `max_value` اگر بازی تمام شده باشد یا عمق به صفر رسیده باشد، تابع ارزیابی صدا زده می‌شود. در غیر این صورت همه حرکت‌های مجاز بازیکن ریشه بررسی می‌شوند و حرکتی انتخاب می‌شود که مقدار بیشتری داشته باشد:

```
1 value = float("-inf")
2 best_move = None
3 for move in ordered_moves(game, legal_moves):
4     child = game.copy()
5     child.make_move(root_player, *move)
6     value2, _ = self.min_value(child, depth - 1, root_player)
7     if value2 > value:
8         value = value2
9     best_move = move
```

تابع `min_value` مشابه همین ساختار را دارد، اما مقدار کمتر را انتخاب می‌کند، چون فرض می‌شود حریف بهترین پاسخ ممکن را برای خودش بازی می‌کند.

۲.۱ عبور از نوبت

در اتللو ممکن است یک بازیکن در نوبت خودش هیچ حرکت مجازی نداشته باشد، اما بازی هنوز تمام نشده باشد. در این حالت نوبت باید به حریف برسد. در کد، اگر بازیکن فعلی حرکت نداشته باشد ولی حریف حرکت داشته باشد، تابع متناظر بازیکن مقابل بدون کم کردن عمق صدا زده می‌شود. اگر هیچ‌کدام از دو بازیکن حرکت نداشته باشند، حالت پایانی است و مقدار ارزیابی برگردانده می‌شود.

۳.۱ عامل Alpha-Beta

الگوریتم Alpha-Beta در فایل `agents/alphabeta_agent.py` پیاده‌سازی شده است. این الگوریتم از نظر نتیجه نظری مانند Minimax است، اما با نگهداری دو کران `alpha` و `beta` بخش‌هایی از درخت را که نمی‌توانند روی تصمیم نهایی اثر بگذارند، حذف می‌کند. در این عامل نیز جستجو با `max_value` شروع می‌شود:

```
1 _, move = self.max_value(
2     game, self.depth, player, float("-inf"), float("inf")
3 )
```

در گره MAX بعد از بررسی هر فرزند، مقدار `alpha` به‌روزرسانی می‌شود. اگر مقدار فعلی از `beta` بزرگ‌تر یا مساوی شود، ادامه فرزندان آن گره بررسی نمی‌شود:

```
1 alpha = max(alpha, value)
2 if value >= beta:
3     return value, best_move
```

در گره MIN هم مقدار `beta` به‌روزرسانی می‌شود و اگر مقدار فعلی از `alpha` کوچک‌تر یا مساوی شود، هرس انجام می‌شود:

```
1 beta = min(beta, value)
2 if value <= alpha:
3     return value, best_move
```

بنابراین Alpha-Beta بدون تغییر دادن مقدار نهایی جستجو، تعداد گره‌های بررسی شده را کاهش می‌دهد.

۲ توضیح تابع ارزیابی

تابع ارزیابی در فایل `agents/search_utils.py` قرار دارد. این تابع همیشه وضعیت بازی را از دید بازیکن ریشه ارزیابی می‌کند، نه از دید بازیکنی که در آن عمق نوبت دارد. بنابراین وقتی در عمق‌های مختلف نوبت بین دو بازیکن عوض می‌شود، مقدار برگشتی همچنان نشان می‌دهد وضعیت برای همان بازیکنی که در ریشه تصمیم می‌گیرد خوب است یا بد. برای حالت پایانی، تابع از نتیجه واقعی بازی استفاده می‌کند. اگر بازیکن برنده باشد مقدار 1000000 و اگر بازنده باشد مقدار 1000000- برگردانده می‌شود. اگر بازی مساوی شود مقدار صفر برگردانده می‌شود:

$$Utility(s, p) = \begin{cases} 1000000 & \text{برنده باشد } p \\ -1000000 & \text{بازنده باشد } p \\ 0 & \text{اگر بازی مساوی باشد} \end{cases}$$

برای حالت‌های غیرپایانی، تابع ارزیابی سبک و قابل فهم نگه داشته شده است تا در تعداد زیاد فراخوانی‌های جستجو کند نشود. فرمول نسخه فعلی کد به شکل زیر است:

$$Eval(s, p) = W_D(s)D(s, p) + M(s, p) + 35C(s, p) + 2E(s, p) - 8CD(s, p) - F(s, p)$$

که در آن:

- $D(s, p)$: اختلاف تعداد مهره‌های بازیکن و حریف.
- $W_D(s)$: وزن اختلاف مهره‌ها بر اساس فاز بازی؛ در ابتدای بازی صفر، در میانه بازی یک، و در انتهای بازی چهار است.
- $M(s, p)$: mobility نرمال شده، یعنی $100 \times \frac{myMoves - oppMoves}{myMoves + oppMoves}$.
- $C(s, p)$: اختلاف تعداد گوشه‌های گرفته شده توسط بازیکن و حریف.
- $E(s, p)$: اختلاف تعداد خانه‌های لبه گرفته شده توسط بازیکن و حریف.
- $CD(s, p)$: تعداد خانه‌های خطرناک کنار گوشه خالی که بازیکن اشغال کرده است، منهای همین مقدار برای حریف.
- $F(s, p)$: اختلاف تعداد مهره‌های مرزی frontier بازیکن و حریف. مهره مرزی مهره‌ای است که حداقل یک خانه خالی در همسایگی خود دارد.

در کد، بخش اصلی تابع ارزیابی به شکل زیر است:

```
1 return (
2     disc_weight * disc_score
3     + mobility_score
4     + 35 * corner_score
5     + 2 * edge_score
6     - 8 * corner_danger_score
7     - frontier_score
8 )
```

۱.۲ دلیل انتخاب عددهای تابع ارزیابی

عدد 1000000 برای حالت‌های پایانی عمداً بسیار بزرگ انتخاب شده است تا برد یا باخت واقعی همیشه از هر تخمین غیرپایانی مهم‌تر باشد. در صفحه 6×6 حد بالای ساده برای قدر مطلق بخش غیرپایانی فقط در حد چند صد واحد است:

$$4 \times 36 + 100 + 35 \times 4 + 2 \times 16 + 8 \times 12 + 36 = 548$$

این عدد فقط یک حد بالای تقریبی و محافظه کارانه است، اما نشان می‌دهد مقدار 1000000 با اختلاف زیادی بیرون از محدوده تابع ارزیابی معمولی قرار دارد. به همین دلیل اگر جستجو به پایان واقعی بازی برسد، عامل هیچ وقت یک موقعیت غیرپایانی ظاهراً خوب را به برد قطعی ترجیح نمی‌دهد.

وزن اختلاف مهره‌ها با تابع `get_disc_weight` بر اساس نسبت خانه‌های پر شده انتخاب می‌شود. اگر کمتر از 0.35 صفحه پر شده باشد، وزن اختلاف مهره‌ها صفر است. در صفحه 6×6 این یعنی تا وقتی حداکثر ۱۲ خانه پر شده باشد، زیاد بودن مهره‌ها امتیاز مستقیمی ندارد. علت این تصمیم این است که در اوایل اتللو، داشتن مهره بیشتر معمولاً به معنی مهره‌های مرزی بیشتر و حرکت‌های امن کمتر است. از ۱۳ تا ۲۶ خانه پر شده، وزن اختلاف مهره‌ها برابر یک می‌شود تا شمارش مهره‌ها کم‌کم وارد تصمیم شود. از ۲۷ خانه به بعد، وزن آن چهار است، چون در آخر بازی نتیجه واقعی بیشتر به اختلاف مهره‌ها نزدیک می‌شود. mobility با ضریب داخلی 100 نرمال شده است و بنابراین همیشه در بازه 100- تا 100 قرار می‌گیرد. به همین دلیل برای mobility_score ضریب جداگانه‌ای در فرمول قرار داده نشده است. این مقیاس باعث می‌شود mobility در ابتدای بازی و میانه بازی اثر جدی داشته باشد، اما همچنان یک گوشه یا حالت پایانی بتواند از آن مهم‌تر باشد.

وزن گوشه‌ها برابر 35 انتخاب شده است، چون گوشه در اتللو پس از گرفته شدن دیگر برگردانده نمی‌شود. در صفحه 6×6 تعداد خانه‌های لبه غیرگوشه ۱۶ است و با وزن 2 بیشترین سهم لبه‌ها حدود 32 می‌شود بنابراین یک گوشه به تنهایی کمی از کل برتری لبه‌ها مهم‌تر است. این نسبت باعث می‌شود عامل گرفتن گوشه را به عنوان یک هدف پایدار و بلندمدت ببیند، نه فقط یک امتیاز کوچک. وزن لبه‌ها برابر 2 است. خانه‌های لبه از خانه‌های داخلی پایدارترند، اما مانند گوشه‌ها کاملاً امن نیستند. بنابراین این عدد بیشتر نقش راهنمای کمی و شکستن تساوی را دارد و اجازه نمی‌دهد عامل فقط به خاطر چند مهره روی لبه، mobility یا خطر دادن گوشه را نادیده بگیرد.

جریمه خانه‌های کنار گوشه خالی با ضریب 8 اعمال می‌شود. برای هر گوشه خالی، سه خانه خطرناک کنار آن بررسی می‌شود. اگر این خانه‌ها متعلق به بازیکن باشند، امتیاز کم می‌شود و اگر متعلق به حریف باشند، از دید بازیکن بهتر است. ضریب ۸ یعنی یک خانه خطرناک کنار گوشه به اندازه چهار خانه لبه معمولی هزینه دارد. این عدد از وزن گوشه کمتر است، چون گرفتن واقعی گوشه هنوز مهم‌تر است، اما آن قدر بزرگ هست که عامل بی‌دلیل در خانه‌هایی مثل X-square و C-square کنار گوشه خالی بازی نکند. امتیاز مهره‌های مرزی بدون ضریب جداگانه و با علامت منفی وارد فرمول شده است: `frontier_score` - مهره مرزی چون کنار خانه خالی قرار دارد، معمولاً بیشتر در معرض برگردانده شدن است و به حریف امکان مانور می‌دهد. ضریب آن یک نکه داشته شده است، چون تعداد مهره‌های مرزی می‌تواند زیاد شود و اگر وزن بزرگی می‌گرفت، عامل بیش از حد دفاعی می‌شد. در نسخه فعلی، خانه‌های خطرناک نزدیک لبه هنوز برای مرتب‌سازی حرکت‌ها استفاده می‌شوند، اما در مقدار نهایی تابع ارزیابی ضریب جداگانه ندارند.

۲.۲ مرتب‌سازی حرکت‌ها

برای قطعی بودن خروجی و کمک به هرس در Alpha-Beta حرکت‌ها قبل از جستجو مرتب می‌شوند. ترتیب فعلی چنین است: حرکت‌های گوشه، حرکت‌های لبه، حرکت‌های معمولی، خانه‌های خطرناک نزدیک لبه، و در آخر خانه‌های خطرناک کنار گوشه خالی. در حالت تساوی نیز ردیف و ستون حرکت استفاده می‌شود تا خروجی تکرارپذیر باشد.

```

1 def get_move_priority(move, corners, edges, edge_danger_squares, corner_danger_squares):
2     if move in corners:
3         return 0
4     if move in corner_danger_squares:
5         return 4
6     if move in edges:
7         return 1
8     if move in edge_danger_squares:
9         return 3
10    return 2

```

۳.۲ جستجوی کامل در انتهای بازی

در هر دو عامل MinimaxAgent و AlphaBetaAgent اگر تعداد خانه‌های خالی صفحه کم باشد، عامل حتی بعد از رسیدن به عمق صفر نیز تا پایان بازی جستجو را ادامه می‌دهد. در تنظیم فعلی، اگر حداکثر ۶ خانه خالی باقی مانده باشد، جستجو تا حالت پایانی ادامه پیدا می‌کند:

```

1 def should_search_to_end(self, game):
2     return self.empty_count(game) <= self.endgame_empty

```

این کار باعث می‌شود تصمیم‌های آخر بازی به جای تخمین، بر اساس نتیجه قطعی بازی گرفته شوند. هزینه آن افزایش مقداری از زمان و تعداد گره‌ها در اواخر بازی است، اما چون فقط نزدیک پایان بازی فعال می‌شود، برای صفحه 6×6 قابل اجرا باقی می‌ماند. این افزایش هزینه در Minimax بیشتر دیده می‌شود، چون برخلاف Alpha-Beta هرس انجام نمی‌دهد.

۳ نتایج آزمایش‌ها

برای اجرای آزمایش‌ها، فایل `run_experiments.py` نوشته شده است. این فایل بدون آرگومان اجرا می‌شود و همه آزمایش‌ها را انجام می‌دهد:

```

1 python3 AI26-PRJ3/run_experiments.py

```

در تنظیمات فعلی، صفحه 6×6 است، برای هر آزمایش ۲۰ بازی انجام می‌شود، و رنگ‌ها بین سیاه و سفید تقسیم می‌شوند. برای جلوگیری از تکرار شدن یک بازی کاملاً یکسان بین عامل‌های قطعی، در ابتدای هر بازی ۲ حرکت تصادفی انجام می‌شود و سپس عامل‌ها بازی را ادامه می‌دهند. عمق‌های ۲، ۳ و ۴ برای Minimax و عمق‌های ۲، ۳، ۴ و ۵ برای Alpha-Beta در برابر RandomAgent و GreedyAgent اجرا شده‌اند. خروجی کامل در فایل `experiment_results.csv` ذخیره شده است.

agent	depth	opponent	W-L-D	win%	diff	move ms	nodes	runtime
minimax	2	random	20-0-0	100	22.20	12.97	336.0	4.02s
minimax	3	random	20-0-0	100	20.85	47.06	874.2	14.55s
minimax	4	random	20-0-0	100	21.90	339.30	7444.8	105.34s
alphabeta	2	random	20-0-0	100	19.75	3.22	55.8	1.05s
alphabeta	3	random	20-0-0	100	24.65	12.03	164.6	3.60s
alphabeta	4	random	20-0-0	100	25.45	28.56	491.0	9.43s
alphabeta	5	random	20-0-0	100	18.70	98.50	1454.8	29.92s
minimax	2	greedy	20-0-0	100	18.30	9.40	223.6	3.01s
minimax	3	greedy	20-0-0	100	19.00	49.20	982.8	16.24s
minimax	4	greedy	20-0-0	100	24.30	213.01	4346.9	67.87s
alphabeta	2	greedy	20-0-0	100	18.30	2.67	43.9	0.90s
alphabeta	3	greedy	20-0-0	100	19.00	11.28	160.5	3.74s
alphabeta	4	greedy	20-0-0	100	24.30	23.27	374.0	7.47s
alphabeta	5	greedy	20-0-0	100	22.00	84.22	1233.8	25.70s

۴ جدول مقایسه Minimax و Alpha-Beta

در عمق برابر و با تابع برش یکسان، انتظار داریم Alpha-Beta همان تصمیم نظری Minimax را برگرداند، اما تعداد گره‌های کمتری را بررسی کند. در نسخه فعلی، جستجوی کامل انتهای بازی برای هر دو عامل فعال است بنابراین تفاوت اصلی در جدول زیر ناشی از هرس Alpha-Beta است.

opponent	depth	minimax nodes	alphabeta nodes	node speedup	time speedup
random	2	336.0	55.8	6.03x	4.03x
random	3	874.2	164.6	5.31x	3.91x
random	4	7444.8	491.0	15.16x	11.88x
greedy	2	223.6	43.9	5.09x	3.51x
greedy	3	982.8	160.5	6.12x	4.36x
greedy	4	4346.9	374.0	11.62x	9.15x

نتایج نشان می‌دهند که اثر هرس در عمق‌های بالاتر واضح‌تر می‌شود. برای نمونه، در برابر GreedyAgent و در عمق ۴، میانگین گره‌های Minimax حدود 4346.9 و میانگین گره‌های Alpha-Beta حدود 374.0 بوده است. از نظر زمان نیز میانگین هر حرکت از 213.01ms به 23.27ms رسیده است.

۵ تحلیل نتایج

۱.۵ اثر افزایش عمق جستجو

افزایش عمق جستجو معمولاً کیفیت تصمیم‌ها را بهتر می‌کند، چون عامل می‌تواند پیامدهای چند حرکت آینده را هم ببیند. در نتایج، هر دو عامل در بیشتر تنظیم‌ها نرخ برد بسیار بالایی داشتند. برای Alpha-Beta افزایش عمق تا ۵ همچنان قابل اجرا بود. در برابر RandomAgent میانگین اختلاف امتیاز از 19.75 در عمق ۲ به 25.45 در عمق ۴ رسید و نرخ برد نیز به ۱۰۰ درصد افزایش پیدا کرد.

این اثر همیشه کاملاً یکنواخت نیست. برای نمونه در بعضی آزمایش‌ها عمق ۵ اختلاف امتیاز کمتری از عمق ۴ داشته است. دلیل آن این است که تابع ارزیابی هنوز تقریبی است، در ابتدای هر بازی چند حرکت تصادفی انجام می‌شود، و بعضی حرکت‌های عمیق‌تر ممکن است با معیارهای فعلی تابع ارزیابی بیش از حد یا کمتر از حد ارزش‌گذاری شوند. با این حال، به طور کلی عمق بیشتر باعث بازی مطمئن‌تر می‌شود.

۲.۵ اثر Alpha-Beta بر زمان اجرا

Alpha-Beta با حذف شاخه‌هایی که نمی‌توانند جواب نهایی را تغییر دهند، زمان اجرا را کم کرده است. این کاهش در عمق ۴ کاملاً مشخص است. در برابر RandomAgent و عمق ۴، میانگین زمان هر حرکت در Minimax برابر 339.30ms و در Alpha-Beta برابر 28.56ms بوده است. در برابر GreedyAgent و عمق ۴ نیز زمان از 213.01ms به 23.27ms رسیده است. مرتب‌سازی حرکت‌ها هم به این موضوع کمک می‌کند، چون اگر حرکت‌های خوب‌تر زودتر بررسی شوند، مقدارهای alpha و beta زودتر محدود می‌شوند و شاخه‌های بیشتری حذف می‌شوند.

۳.۵ معیارهای تابع ارزیابی

در تابع ارزیابی فعلی از معیارهای زیر استفاده شده است:

- اختلاف تعداد مهره‌ها با وزن وابسته به فاز بازی.
- mobility نرمال‌شده بر اساس تعداد حرکت‌های مجاز دو طرف.

- مالکیت گوشه‌ها با وزن زیاد.
- مالکیت خانه‌های لبه با وزن متوسط.
- جریمه خانه‌های کنار گوشه خالی.
- جریمه مهره‌های مرزی که کنار خانه‌های خالی قرار دارند.

۴.۵ مهم‌ترین عامل موفقیت عامل

مهم‌ترین عامل موفقیت عامل، ترکیب، *mobility* گوشه‌ها، جلوگیری از دادن گوشه به حریف، و کم کردن مهره‌های مرزی است. فقط نگاه کردن به تعداد مهره‌ها در اوایل بازی خوب نیست، چون ممکن است بازیکن مهره‌های زیادی داشته باشد ولی حرکت‌های امن کمی برای ادامه بازی باقی بماند. *mobility* باعث می‌شود عامل موقعیت‌هایی را ترجیح دهد که گزینه‌های بیشتری در آینده دارند. وزن زیاد گوشه‌ها نیز باعث می‌شود عامل به خانه‌هایی توجه کند که در ادامه بازی قابل برگرداندن نیستند. جریمه مهره‌های مرزی هم عامل را از ساختن موقعیت‌های خیلی شکننده دور می‌کند.

از طرف دیگر، *Alpha-Beta* امکان استفاده از عمق بیشتر را فراهم می‌کند. به همین دلیل عامل می‌تواند با هزینه زمانی قابل قبول‌تر، پیامدهای چند حرکت آینده را بررسی کند.

۵.۵ نقاط ضعف تابع ارزیابی

تابع ارزیابی فعلی ساده و سریع است، اما کامل نیست. در آن پایداری واقعی مهره‌ها محاسبه نمی‌شود و فقط یک تقریب ساده از پایداری با گوشه‌ها، لبه‌ها و مهره‌های مرزی داریم. همچنین خانه‌های خطرناک نزدیک لبه در مقدار نهایی تابع ارزیابی ضریب جداگانه ندارند و فقط در مرتب‌سازی حرکت‌ها استفاده می‌شوند. وزن‌ها نیز به صورت دستی تنظیم شده‌اند و ممکن است برای اندازه صفحه یا نوع حریف‌های متفاوت بهترین انتخاب نباشند.

همچنین فقط وزن اختلاف مهره‌ها با فاز بازی تغییر می‌کند. شاید در نسخه‌های بعدی بتوان وزن *mobility* خطر گوشه، لبه و مهره‌های مرزی را هم بر اساس فاز بازی تغییر داد. برای مثال، *mobility* در ابتدای بازی مهم‌تر است، اما در چند حرکت آخر، نتیجه دقیق و اختلاف مهره‌ها اهمیت بیشتری پیدا می‌کند.

جمع‌بندی

در این پروژه دو عامل *MinimaxAgent* و *AlphaBetaAgent* برای بازی اتللو پیاده‌سازی شد. هر دو عامل از جستجوی درخت بازی استفاده می‌کنند و مقدار حالت‌ها را از دید بازیکن ریشه محاسبه می‌کنند. *Alpha-Beta* از نظر نظری با همان عمق و همان تابع برش، همان نتیجه *Minimax* را تولید می‌کند، اما به کمک هرس، تعداد گره‌های بررسی‌شده و زمان اجرا را کاهش می‌دهد. در پیاده‌سازی نهایی، جستجوی کامل انتهای بازی نیز به هر دو عامل *MinimaxAgent* و *AlphaBetaAgent* اضافه شد تا تصمیم‌های آخر بازی دقیق‌تر باشند. نتایج آزمایش‌ها نشان داد که افزایش عمق جستجو معمولاً کیفیت بازی را بهتر می‌کند، اما هزینه زمانی آن زیاد است و استفاده از *Alpha-Beta* برای عملی شدن عمق‌های بالاتر ضروری است.

منابع

برای طراحی ایده‌های تابع ارزیابی اتللو، از بحث زیر نیز استفاده شد:

<https://stackoverflow.com/questions/12334216/othello-evaluation-function>